

SESSION 7 - Aggregate Functions

TASKS

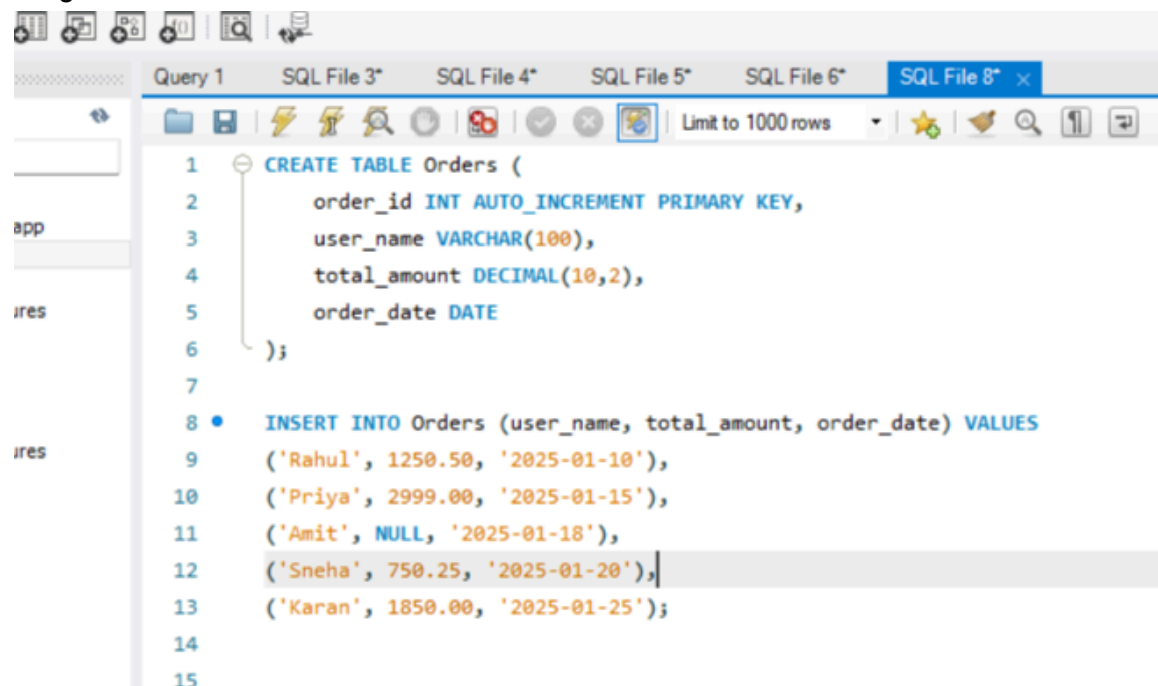
1. Create a table called Orders with columns: order_id, user_name, total_amount, and order_date. Insert 5 sample rows with different users and order amounts, including at least one NULL value for total_amount.

Query / Answer:

```
CREATE TABLE Orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_name VARCHAR(100),  
    total_amount DECIMAL(10,2),  
    order_date DATE  
);
```

```
INSERT INTO Orders (user_name, total_amount, order_date) VALUES  
( 'Rahul', 1250.50, '2025-01-10'),  
( 'Priya', 2999.00, '2025-01-15'),  
( 'Amit', NULL, '2025-01-18'),  
( 'Sneha', 750.25, '2025-01-20'),  
( 'Karan', 1850.00, '2025-01-25');
```

Image / Screenshot:



2. Write a SQL query to count how many orders were placed by each user in the Orders table, displaying user_name and the number of orders as order_count.

Query / Answer:

```
CREATE TABLE Orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_name VARCHAR(100),  
    total_amount DECIMAL(10,2),  
    order_date DATE  
);
```

```
INSERT INTO Orders (user_name, total_amount, order_date) VALUES  
('Rahul', 1250.50, '2025-01-10'),  
('Priya', 2999.00, '2025-01-15'),  
('Amit', NULL, '2025-01-18'),  
('Sneha', 750.25, '2025-01-20'),  
('Karan', 1850.00, '2025-01-25');
```

```
select user_name ,  
count(*) as Total_items from Orders group by user_name;
```

Image / Screenshot:

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
13 ('Karan', 1850.00, '2025-01-25');  
14  
15  
16 select user_name ,  
17 count(*) as Total_items from Orders group by user_name;  
18  
19
```

The results grid displays the output of the query, showing the user_name and the total number of items (Total_items) for each user.

user_name	Total_items
Rahul	1
Priya	1
Amit	1
Sneha	1
Karan	1

--

3. Write a SQL query to calculate the average total_amount of all orders in the Orders table, making sure to ignore any NULL values.

Query / Answer:

select avg(total_amount) as Gross from Orders

Image / Screenshot:

The screenshot shows a SQL IDE interface. The query editor on the left contains the following SQL code:

```
13 ('Karan', 1850.00, '2025-01-25');
14
15
16 • select user_name ,
17     count(*) as Total_items from Orders group by user_name;
18
19 • select avg(total_amount) as Gross from Orders
```

The query editor has a sidebar on the left with labels 'pp' and 'res'. The query is highlighted in blue. Below the query editor is a toolbar with buttons for 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' button is active, and the results are displayed in a table below the toolbar.

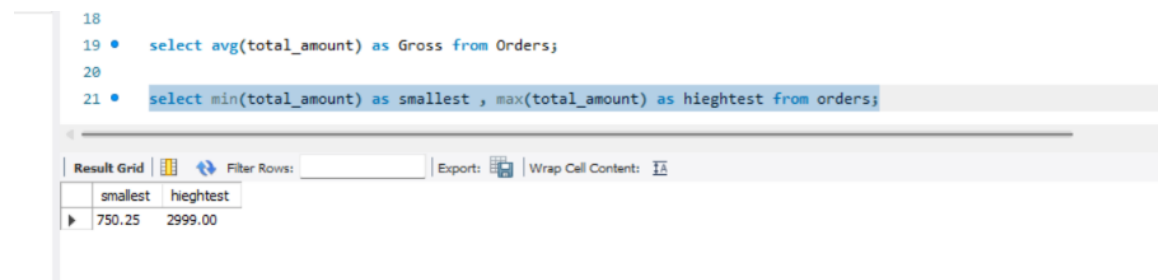
Gross
1712.437500

4. Suppose you are building a Flipkart-style dashboard: Write a SQL query to find the highest and lowest order amounts (MAX and MIN) from the Orders table, and display both values in a single result row.

Query / Answer:

```
select min(total_amount) as smallest , max(total_amount) as hieghtest from orders;
```

Image / Screenshot:



5. Write a SQL query to calculate the total sales (SUM of total_amount) for all orders, but only include orders where total_amount is not NULL. Hint: Use a WHERE clause to filter out NULL values before applying the SUM function.

Query / Answer:

```
select sum(total_amount) as gross from orders where total_amount is not null;
```

Image / Screenshot:

